

FRANCIS XAVIER ENGINEERING COLLEGE, TIRUNELVELI
(An Autonomous Institution)

DEPARTMENT OF INFORMATION TECHNOLOGY

ACADEMIC YEAR: 2026-27/ODD

BATCH: 2025 – 29

SEM: 03

COURSE CODE | NAME : 24IT3602 JAVA PROGRAMMING

QUESTION BANK

UNIT I - OOP CONCEPTS & BASICS OF JAVA

History of JAVA – solid principles - Object Oriented Programming Fundamentals –Structure of Java Program, Data Types, Variables, and Arrays - Operators

Introducing Classes - Methods and Classes, Constructor and Destructor - Finalize method.

I/O Basics - Reading Console Input -Writing Console output- Control Statements.

Topic Distribution (T1–T6)

- T1: OOP Fundamentals & SOLID Principles
- T2: Data Types, Variables, Arrays & Operators
- T3: Structure of a Java Program & Classes
- T4: Methods, Constructors & finalize()
- T5: I/O Basics (Console Input/Output)
- T6: Control Statements

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
1	A billing module class Invoice has methods calculateTax(), printInvoice(), and emailCustomer() all inside one class, so any change to email formatting forces recompilation of tax logic. Identify which SOLID principle is violated and redesign the class structure (name the resulting classes) to fix it. Answer The violated SOLID principle is the Single Responsibility Principle (SRP). The Invoice class has three different responsibilities: ● Tax calculation ● Invoice printing/formatting ● Customer email communication A change in email formatting should not require changes or recompilation of tax-related logic. Redesigned class structure <pre> Invoice +-- InvoiceTaxCalculator +-- InvoicePrinter +-- CustomerEmailService </pre> Resulting classes:	5	T1	CO1	K4	PI 04.03.01

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
	1. Invoice – Holds invoice data and core invoice information. 2. InvoiceTaxCalculator – Responsible only for calculating tax. 3. InvoicePrinter – Responsible only for printing/formatting the invoice. 4. CustomerEmailService – Responsible only for emailing the customer. Example <pre> class Invoice { // Invoice data } class InvoiceTaxCalculator { double calculateTax(Invoice invoice) { // Tax calculation logic return 0; } } class InvoicePrinter { void printInvoice(Invoice invoice) { // Printing/formatting logic } } class CustomerEmailService { void emailCustomer(Invoice invoice) { // Email logic } } </pre>					
2	Two designs are shown for a Shape hierarchy. Design A computes area with a chain of instanceof checks inside one method, requiring modification whenever a new shape is added. Design B defines an abstract area() method overridden by each subclass. Analyze which design violates the Open-Closed Principle and justify using the two code excerpts. <pre> // Design A double area(Object s){ if(s instanceof Circle) return 3.14*((Circle)s).r*((Circle)s).r; else if(s instanceof Square) return ((Square)s).side*((Square)s).side; // must edit this method for every new shape </pre>	5	T1	CO 1	K4	PI 04.03.0 1

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
	<pre> } // Design B abstract class Shape{ abstract double area(); } class Circle extends Shape{ double area(){ return 3.14*r*r; } } </pre> Answer Design A violates the Open-Closed Principle. The Open-Closed Principle states that a software entity should be open for extension but closed for modification. Design A — Violates OCP <pre> double area(Object s){ if(s instanceof Circle) return 3.14 * ((Circle)s).r * ((Circle)s).r; else if(s instanceof Square) return ((Square)s).side * ((Square)s).side; // must edit this method for every new shape } </pre> If a new shape such as Triangle is added, the area() method must be modified: <pre> else if(s instanceof Triangle) // calculate triangle area </pre> Thus, the existing method is not closed for modification. Every new shape requires changes to existing code. Design B — Follows OCP <pre> abstract class Shape { abstract double area(); } class Circle extends Shape { double area() { return 3.14 * r * r; } } </pre> Here, each shape provides its own implementation of area(). To add a new shape, we simply create another subclass: <pre> class Square extends Shape { double area() { return side * side; } } class Triangle extends Shape { double area() { </pre>					

S.No	Question	Marks	Topic	CO	RBT Level	PI Code						
	<pre> return 0.5 * base * height; } } </pre> The existing Shape hierarchy and code that uses Shape do not need to be modified. Conclusion <table> <tr> <td>Design OCP</td> <td>Reason</td> </tr> <tr> <td>Design A  Violates</td> <td>instanceof chain must be modified for every new shape</td> </tr> <tr> <td>Design B  Follows</td> <td>New shapes extend Shape and implement area() without modifying existing classes</td> </tr> </table> Therefore, Design A violates OCP, while Design B satisfies OCP through polymorphism and abstraction.	Design OCP	Reason	Design A  Violates	instanceof chain must be modified for every new shape	Design B  Follows	New shapes extend Shape and implement area() without modifying existing classes					
Design OCP	Reason											
Design A  Violates	instanceof chain must be modified for every new shape											
Design B  Follows	New shapes extend Shape and implement area() without modifying existing classes											
3	A BankAccount class exposes its balance field as 'public double balance;' allowing external code to directly set balance to a negative number, e.g. acc.balance = -5000;. Identify the OOP principle broken and rewrite the class declaration (fields/methods) to prevent this. Answer The OOP principle violated is Encapsulation. By declaring balance as public, external code can directly modify it and put the account into an invalid state. Corrected class declaration <pre> class BankAccount { private double balance; public BankAccount(double initialBalance) { if (initialBalance >= 0) this.balance = initialBalance; else this.balance = 0; } public double getBalance() { return balance; } public void deposit(double amount) { if (amount > 0) </pre>	5	T1	CO1	K4	PI 04.03.01						

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
	<pre> balance += amount; } public boolean withdraw(double amount) { if (amount > 0 && amount <= balance) { balance -= amount; return true; } return false; } } </pre> Why this fixes the problem - balance is private, so external code cannot do <code>acc.balance = -5000;</code>. <code>getBalance()</code> provides controlled read access. <code>deposit()</code> and <code>withdraw()</code> provide controlled modification. - Validation ensures balance cannot become negative through these methods. Answer: Encapsulation — achieved using private fields and controlled public methods.					
4.a	Given the hierarchy Vehicle -> Car -> Truck, Truck overrides <code>loadCargo()</code> to throw <code>UnsupportedOperationException</code> because Trucks in this system cannot exceed a load limit, while client code written for Vehicle calls <code>loadCargo()</code> on all vehicle objects uniformly. (a) Name the SOLID principle being violated. Answer (a) SOLID Principle Violated: Liskov Substitution Principle (LSP) The Liskov Substitution Principle states: Objects of a superclass should be replaceable with objects of its subclasses without breaking the correctness of the program. Here, Truck is a subclass of Vehicle, but it overrides <code>loadCargo()</code> by throwing <code>UnsupportedOperationException</code>. <pre> Vehicle v = new Truck(); v.loadCargo(); // Throws UnsupportedOperationException </pre>	2	T1	CO 1	K4	PI 04.03.0 1

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
	The client expects every Vehicle to support loadCargo(), but a Truck cannot fulfill that contract. Therefore, substituting Truck for Vehicle causes the client code to fail. Answer: Liskov Substitution Principle (LSP) is violated.					
4.b	Given the hierarchy Vehicle -> Car -> Truck, Truck overrides loadCargo() to throw UnsupportedOperationException because Trucks in this system cannot exceed a load limit, while client code written for Vehicle calls loadCargo() on all vehicle objects uniformly. Redesign the hierarchy briefly so client code remains safe. Answer The problem is a violation of the Liskov Substitution Principle (LSP). A Truck should not inherit a loadCargo() contract that it cannot safely honor. Safe redesign Separate vehicles that support cargo loading from general vehicles: <pre> abstract class Vehicle { // Common vehicle behavior } interface CargoCarrier { void loadCargo(double weight); } class Car extends Vehicle { // Car-specific behavior } class Truck extends Vehicle implements CargoCarrier { private double loadLimit; public void loadCargo(double weight) { if (weight <= loadLimit) { // Load cargo } } } </pre> Client code should depend on the appropriate abstraction: <pre> void loadVehicle(CargoCarrier vehicle, double weight) { vehicle.loadCargo(weight); } </pre>	3	T1	CO 1	K4	PI 04.03.0 1

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
	} Now, only vehicles that actually support cargo loading implement CargoCarrier. The client does not call an unsupported method, and no subclass needs to throw UnsupportedOperationException. Principle applied: Liskov Substitution Principle (LSP).					
5	A PaymentProcessor class directly creates objects of CreditCardPayment, UPIPayment, and NetBankingPayment using 'new' inside its process() method, so adding a new payment mode requires editing PaymentProcessor. Identify the violated principle and propose an interface-based redesign. Answer The violated SOLID principle is the Dependency Inversion Principle (DIP). PaymentProcessor directly depends on concrete classes such as CreditCardPayment, UPIPayment, and NetBankingPayment. Adding a new payment mode requires modifying PaymentProcessor. Interface-based redesign <pre> interface Payment { void pay(double amount); } class CreditCardPayment implements Payment { public void pay(double amount) { System.out.println("Paid using Credit Card"); } } class UPIPayment implements Payment { public void pay(double amount) { System.out.println("Paid using UPI"); } } class NetBankingPayment implements Payment { public void pay(double amount) { System.out.println("Paid using Net Banking"); } } </pre>	5	T1	CO 1	K4	PI 04.03.0 1

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
	<pre> } class PaymentProcessor { private Payment payment; PaymentProcessor(Payment payment) { this.payment = payment; } void process(double amount) { payment.pay(amount); } } </pre> Usage PaymentProcessor p1 = new PaymentProcessor(new CreditCardPayment()); PaymentProcessor p2 = new PaymentProcessor(new UPIPayment()); p1.process(500); p2.process(1000); To add a new payment mode, such as WalletPayment, only a new class implementing Payment is required. PaymentProcessor remains unchanged. Answer: Dependency Inversion Principle (DIP) — depend on the Payment abstraction, not concrete payment classes.					
6	Trace the code below, state the printed output, identify why it is wrong for computing an average, and correct the code. <pre> int total = 275, count = 6; double avg = total / count; System.out.println(avg); </pre> Answer Since both total and count are integers, total / count performs integer division first: 275 / 6 = 45 (remainder discarded) Then 45 is assigned to double avg, giving 45.0. Printed output 45.0 Why is it wrong? The actual average is 45.8333..., but integer division truncates the decimal portion before assigning the result to double. Correct code	5	T2	CO 1	K4	PI 04.03.0 1

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
	<pre>int total = 275, count = 6; double avg = (double) total / count; System.out.println(avg);</pre> Correct output 45.833333333333336 Key point: At least one operand must be converted to double before division.					
7	Given a 4x4 seating matrix <code>int[][] seat = new int[4][4]</code>; a snippet marks every seat whose (row+col) is even using <code>seat[r][c]=1</code> inside nested loops (rows/cols numbered 0-3). (a) [3m] Trace the resulting 4x4 matrix of 0s/1s. (b) [2m] Rewrite the condition using a bitwise operator instead of %. <pre>for(int r=0;r<4;r++) for(int c=0;c<4;c++) if((r+c)%2==0) seat[r][c]=1;</pre> Answer (a) Resulting 4×4 Matrix The condition <code>(r + c) % 2 == 0</code> marks positions where the sum of row and column indices is even. <pre>1 0 1 0 0 1 0 1 1 0 1 0 0 1 0 1</pre> (b) Using a Bitwise Operator For checking whether a number is even, use the bitwise AND (&) operator: <pre>for(int r = 0; r < 4; r++) for(int c = 0; c < 4; c++) if(((r + c) & 1) == 0) seat[r][c] = 1;</pre> Reason: The least significant bit of an even number is 0, so number & 1 equals 0 for even numbers.	5	T2	CO ₁	K4	PI 04.03.0 1
8	The table below shows four declarations. Identify which line(s) fail to compile, explain why, and state the corrected literal/type for each faulty line. <pre>byte b = 130; // Line 1 short s = 40000; // Line 2 int i = 2147483648L; // Line 3 (note the L) long l = 100; // Line 4</pre> Answer	5	T2	CO ₁	K4	PI 04.03.0 1

S.No	Question	Marks	Topic	CO	RBT Level	PI Code																				
	Analysis of Each Line <table> <tr> <th>Line</th> <th>Declaration</th> <th>Compiles ?</th> <th>Reason / Correction</th> </tr> <tr> <td>1</td> <td>byte b = 130;</td> <td>✗ No</td> <td>byte range is -128 to 127. 130 is out of range.</td> </tr> <tr> <td>2</td> <td>short s = 40000;</td> <td>✗ No</td> <td>short range is -32,768 to 32,767. 40000 is out of range.</td> </tr> <tr> <td>3</td> <td>int i = 2147483648L;</td> <td>✗ No</td> <td>2147483648L is a long value and is larger than the maximum int value.</td> </tr> <tr> <td>4</td> <td>long l = 100;</td> <td>✓ Yes</td> <td>Integer literal 100 can be safely widened from int to long.</td> </tr> </table> Corrected declarations byte b = 127; // Line 1 short s = 32767; // Line 2 long i = 2147483648L; // Line 3 long l = 100; // Line 4 Key ranges: - byte: -128 to 127 - short: -32,768 to 32,767 - int: -2,147,483,648 to 2,147,483,647 - long: -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807	Line	Declaration	Compiles ?	Reason / Correction	1	byte b = 130;	✗ No	byte range is -128 to 127. 130 is out of range.	2	short s = 40000;	✗ No	short range is -32,768 to 32,767. 40000 is out of range.	3	int i = 2147483648L;	✗ No	2147483648L is a long value and is larger than the maximum int value.	4	long l = 100;	✓ Yes	Integer literal 100 can be safely widened from int to long.					
Line	Declaration	Compiles ?	Reason / Correction																							
1	byte b = 130;	✗ No	byte range is -128 to 127. 130 is out of range.																							
2	short s = 40000;	✗ No	short range is -32,768 to 32,767. 40000 is out of range.																							
3	int i = 2147483648L;	✗ No	2147483648L is a long value and is larger than the maximum int value.																							
4	long l = 100;	✓ Yes	Integer literal 100 can be safely widened from int to long.																							
9	The output produced differs slightly from a hand-calculated compound-interest value obtained using double throughout. Analyze why the float usage introduces precision loss and rewrite the line to eliminate it. <pre>float rate = 0.1f; double principal = 100000; int years = 3; double amount = principal * (float)Math.pow((1+rate), years); System.out.println(amount);</pre>	5	T2	CO1	K4	PI 04.03.01																				

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
	Answer The precision loss occurs because rate is declared as a float: float rate = 0.1f; 0.1 cannot be represented exactly in binary floating-point. A float has only 32-bit precision, so 0.1f is stored as an approximation. That approximation is then used in: (float)Math.pow((1 + rate), years) There are two precision-loss points: 1. rate is stored as a float. 2. Math.pow() returns a double, but the result is explicitly cast back to float. Corrected version Use double throughout: double rate = 0.1; double principal = 100000; int years = 3; double amount = principal * Math.pow(1 + rate, years); System.out.println(amount); The calculated value is approximately: 133100.0 Key point Change: float rate = 0.1f; double amount = principal * (float)Math.pow((1+rate), years); to: double rate = 0.1; double amount = principal * Math.pow(1 + rate, years); This avoids the unnecessary narrowing conversion from double to float and maintains double precision throughout the calculation.					
10	An eligibility check is written as boolean eligible = (marks>=40 attendance>=75) && (marks>=40 && attendance>=75); intended to mean 'pass only if BOTH marks and attendance criteria are met.' For marks=35, attendance=80, the program still evaluates the outer unnecessarily. Simplify the expression to a logically equivalent, minimal form and justify using the given values.	5	T2	CO 1	K4	PI 04.03.0 1

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
	Answer The original expression is: boolean eligible = (marks >= 40 attendance >= 75) && (marks >= 40 && attendance >= 75); Simplification Let: - A = marks >= 40 - B = attendance >= 75 Then the expression is: (A B) && (A && B) Using the Boolean identity: (A B) && (A && B) = A && B So the minimal equivalent expression is: boolean eligible = marks >= 40 && attendance >= 75; For the given values marks = 35 attendance = 80 Therefore: marks >= 40 → false attendance >= 75 → true Original expression: (false true) && (false && true) true && false = false Simplified expression: false && true = false So the student is not eligible. Why simplify? The (marks >= 40 attendance >= 75) part is redundant because the second condition already requires both conditions to be true. The minimal and clearer form is: boolean eligible = marks >= 40 && attendance >= 75;					
11	Saved as StudentRecord.java and run with 'java StudentRecord', this produces a runtime error 'Main method not found'. Identify every deviation from the required main-method signature and rewrite the corrected method header. <pre>class StudentRecord { void Main(String[] args) { System.out.println("Started"); } }</pre>	5	T3	CO ₁	K4	PI 04.03.0 1

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
	<pre> } } </pre> Answer There are two deviations from Java's required main method signature: 1. Main is written with an uppercase M. Java is case-sensitive; it must be main. 2. The method is declared as void Main(...) instead of the required public static void main(...). The corrected method header is: public static void main(String[] args) So the corrected program is: class StudentRecord { public static void main(String[] args) { System.out.println("Started"); } } Why the runtime error occurs: When you run java StudentRecord, the JVM specifically searches for a method with the exact signature public static void main(String[] args). Since Main is neither lowercase main nor static/public, the JVM cannot find the entry point.					
12	Complete the missing method body so the program prints 'Roll:101 Name:Arun'. Write the missing display() body. <pre> class Student { int roll; String name; Student(int r, String n){ roll=r; name=n; } void display(){ /* MISSING */ } public static void main(String[] a){ new Student(101,"Arun").display(); } } </pre> Answer The missing display() body is: void display() { System.out.println("Roll:" + roll + " Name:" + name); } This prints: Roll:101 Name:Arun	5	T3	CO 1	K3	PI 03.02.0 1

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
13	A file Billing.java contains 'public class Bill {...}' and 'class Item {...}'. The developer compiles it as 'javac Bill.java' but the file itself is saved as Billing.java. State whether compilation succeeds, and if not, identify the exact rule violated and the fix. Answer Compilation does not succeed. • Rule violated: A public top-level class must have the same name as the source file. • The file is named Billing.java, but the public class is Bill. • Fix: Rename the file to Bill.java and compile using: javac Bill.java Then Bill and the non-public Item class can coexist in the same file.	5	T3	CO 1	K4	PI 04.03.0 1
14	Given the code below: (a) [2m] State the value of r1.area() after this code executes and explain why. (b) [3m] Modify the code so r2 becomes an independent copy instead of referencing the same object. <pre>class Rectangle { int l, b; Rectangle(int l,int b){ this.l=l; this.b=b; } int area(){ return l*b; } } Rectangle r1 = new Rectangle(4,5); Rectangle r2 = r1; r2.l = 10;</pre> Answer (a) Value of r1.area() r1.area() returns 50. Reason: r2 = r1 does not create a new Rectangle object. Both r1 and r2 refer to the same object. Therefore, when r2.l = 10, the value of l in the shared object becomes 10. So: r1.area() = 10 × 5 = 50 (b) Independent copy Create a new object using the values of r1: Rectangle r2 = new Rectangle(r1.l, r1.b); Now r1 and r2 refer to different objects. Therefore, changing r2.l will not affect r1.l.	5	T3	CO 1	K4	PI 04.03.0 1

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
15	Trace this program and state whether it compiles and runs successfully; justify the accessibility of the private field x from Inner. <pre> class Outer { private int x = 10; class Inner { void show(){ System.out.println(x); } } } public class Test { public static void main(String[] a){ Outer.Inner in = new Outer().new Inner(); in.show(); } } </pre>	5	T3	CO ₁	K4	PI 04.03.01
16	One statement in main() fails to compile. Identify which line and why, then correct it so the program compiles, and trace the final printed output. <pre> class Box { Box(){ System.out.println("Default"); } Box(int s){ System.out.println("Side "+s); } Box(int l,int b){ System.out.println("L,B "+l+", "+b); } } public class Test{ public static void main(String[] a){ new Box(); new Box(5); new Box(5,10.0); } } </pre> Answer The statement that fails to compile is: new Box(5,10.0); Why? The available constructors are: - Box() → no arguments - Box(int s) → one int argument - Box(int l, int b) → two int arguments But 10.0 is a double literal, so it cannot be passed directly to the int parameter. Corrected statement new Box(5,10); Final output Default	5	T4	CO ₁	K4	PI 04.03.01

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
	Side 5 L,B 5,10 Thus, the program compiles successfully after changing 10.0 to 10.					
17	Given three overloaded display() methods (int, long, Integer) and the call display(10);, state which overload is invoked, applying Java's overload resolution order: exact match > widening > autoboxing. <pre>void display(int x){ System.out.println("int "+x); } void display(long x){ System.out.println("long "+x); } void display(Integer x){ System.out.println("Integer "+x); } ... display(10);</pre> Answer The overload invoked is: int 10 Reason: 10 is an integer literal of type int. Therefore, display(int x) is an exact match and is selected before widening to long or autoboxing to Integer. Overload resolution order: Exact match (int) > Widening (long) > Autoboxing (Integer).	5	T4	CO 1	K4	PI 04.03.0 1
18	(a) [3m] Complete the MISSING line using constructor chaining (this(...)) so new Employee(5) also sets name to "Unassigned". (b) [2m] State the output of System.out.println(new Employee(5).name); after your fix. <pre>class Employee{ int id; String name; Employee(int id){ this.id=id; /* MISSING */ } Employee(int id, String name){ this.id=id; this.name=name; } }</pre> Answer There is an important point: this(...) must be the first statement in a constructor, so it cannot be used after this.id = id.	5	T4	CO 1	K3	PI 03.02.0 1

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
	(a) Corrected constructor using constructor chaining <pre>Employee(int id){ this(id, "Unassigned"); }</pre> The complete class becomes: <pre>class Employee { int id; String name; Employee(int id) { this(id, "Unassigned"); } Employee(int id, String name) { this.id = id; this.name = name; } }</pre> (b) Output Unassigned Reason: new Employee(5) calls the one-argument constructor, which chains to Employee(5, "Unassigned"), setting name to "Unassigned".					
19	Trace the exact order of output for both object creations, explaining when the instance initializer block executes relative to each constructor. <pre>class Counter{ int val; { val = 5; System.out.println("Init block"); } Counter(){ System.out.println("Constructor, val="+val); } Counter(int v){ val = v; System.out.println("Param constructor, val="+val); } } new Counter(); new Counter(20);</pre> Answer The instance initializer block executes before the constructor body every time an object is created. Exact output order Init block	5	T4	CO 1	K4	PI 04.03.0 1

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
	Constructor, val=5 Init block Param constructor, val=20 Trace 1. new Counter(); - Instance initializer executes first → val = 5 - Prints Init block - Default constructor executes - Prints Constructor, val=5 2. new Counter(20); - Instance initializer executes first → val = 5 - Prints Init block - Parameterized constructor executes → val = 20 - Prints Param constructor, val=20 Thus, the initializer block runs once for each object, immediately before its constructor body.					
20	For input "22\nRahul\n", trace execution and state the actual printed value of name, explaining the cause of the skipped input. Scanner sc = new Scanner(System.in); int age = sc.nextInt(); String name = sc.nextLine(); System.out.println("Name:"+name+"Age:"+age); Answer For the input: 22 Rahul Trace - sc.nextInt() reads 22 but does not consume the newline (\n) after 22. - sc.nextLine() then reads the remaining part of the current line, which is just the empty string. - Therefore, "Rahul" is not stored in name. Actual output Name: Age:22 Cause: nextInt() leaves the newline character in the input buffer, and nextLine() consumes that leftover newline instead of reading "Rahul".	5	T5	CO 1	K4	PI 04.03.0 1

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
21	Complete the missing statement(s) inside the loop so the program correctly sums n integers entered on separate lines. <pre>BufferedReader br = new BufferedReader(new InputStreamReader(System.in)); int n = Integer.parseInt(br.readLine()); int sum = 0; for(int i=0;i<n;i++){ /* MISSING: read one integer per line and add to sum */ }</pre> System.out.println(sum); Answer Use readLine() to read each integer and Integer.parseInt() to convert it to an int: sum += Integer.parseInt(br.readLine()); So the completed loop is: <pre>for(int i = 0; i < n; i++) { sum += Integer.parseInt(br.readLine()); }</pre> This reads one integer per line and adds each value to sum.	5	T5	CO ₁	K3	PI 03.02.0 1
22	Trace and write the exact console output layout (including line breaks) produced by this sequence. <pre>System.out.print("Result: "); System.out.print(10); System.out.println(20); System.out.println(30);</pre> Answer he exact console output is: Result: 1020 30 Trace • print("Result: ") → prints text without a line break. • print(10) → prints 10 on the same line. • println(20) → prints 20 and then moves to the next line. • println(30) → prints 30 and moves to the next line.	5	T5	CO ₁	K4	PI 04.03.0 1

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
23	(a) [2m] Explain the role of the extra <code>sc.nextLine()</code> after <code>nextInt()</code> in this snippet. (b) [3m] Predict what goes wrong for input "3\nAnu" if that line is removed, tracing the value stored in name. <pre>Scanner sc = new Scanner(System.in); System.out.print("Enter count: "); int n = sc.nextInt(); sc.nextLine(); System.out.print("Enter name: "); String name = sc.nextLine();</pre>	5	T5	CO ₁	K4	PI 04.03.01
24	A flowchart reads: Start -> Input marks(int) -> If marks >= 90 print 'A' -> else if marks >= 75 print 'B' -> else if marks >= 50 print 'C' -> else print 'F' -> End, using console I/O. For input marks=82, identify the exact branch executed and write the complete Java if-else chain that implements ALL branches correctly (not just the true one). Answer For marks = 82, the condition marks >= 90 is false, and marks >= 75 is true. Exact branch executed else if (marks >= 75) → prints B Complete Java if-else chain <pre>if (marks >= 90) { System.out.println("A"); } else if (marks >= 75) { System.out.println("B"); } else if (marks >= 50) { System.out.println("C"); } else { System.out.println("F"); }</pre> For marks = 82, the exact output is: B	5	T5	CO ₁	K4	PI 04.03.01
25	Trace and state the exact output printed. <pre>for(int i=1;i<=5;i++){ if(i==3) continue; if(i==5) break; System.out.print(i+" "); }</pre> Answer The loop executes as follows: i = 1 → prints 1	5	T6	CO ₁	K4	PI 04.03.01

S.No	Question	Marks	Topic	CO	RBT Level	PI Code
	• $i = 2 \rightarrow$ prints 2 • $i = 3 \rightarrow$ continue skips printing • $i = 4 \rightarrow$ prints 4 • $i = 5 \rightarrow$ break stops the loop before printing Exact output 1 2 4					